

Version Control Documentation

Milestones 2-5: ERD, Normalization, Dataset, DDL, and DML

Group Name	[Enter Group Name]
Member 1	Haroon Atif
Member 2	[Enter Second Member Name]
GitHub Repository Link	[Paste GitHub Repository Link Here]
Database Project	Gaming / Tournament Platform Database
PDF Filename	GroupName_Version_Control_DBLab.pdf

Submission checklist covered in this document:

- Updated ERD diagram based on the finalized normalized schema.
- Normalization walkthrough from 1NF to 3NF with justifications.
- Project-specific dataflow description for the gaming/tournament platform.
- Repository contents checklist for CSV, DDL, DML, and validation outputs.
- Required commit-message reminders for Milestones 4 and 5.

Important: Replace the placeholders for group name, second member name, and GitHub repository link before submitting. Ensure both group members have commits in the repository.

Updated ERD Diagram

The final ERD reflects the normalized schema through 3NF. It shows primary keys, foreign keys, junction tables, and cardinalities for tournaments, games, players, social groups, chats, messages, leaderboards, and notifications.

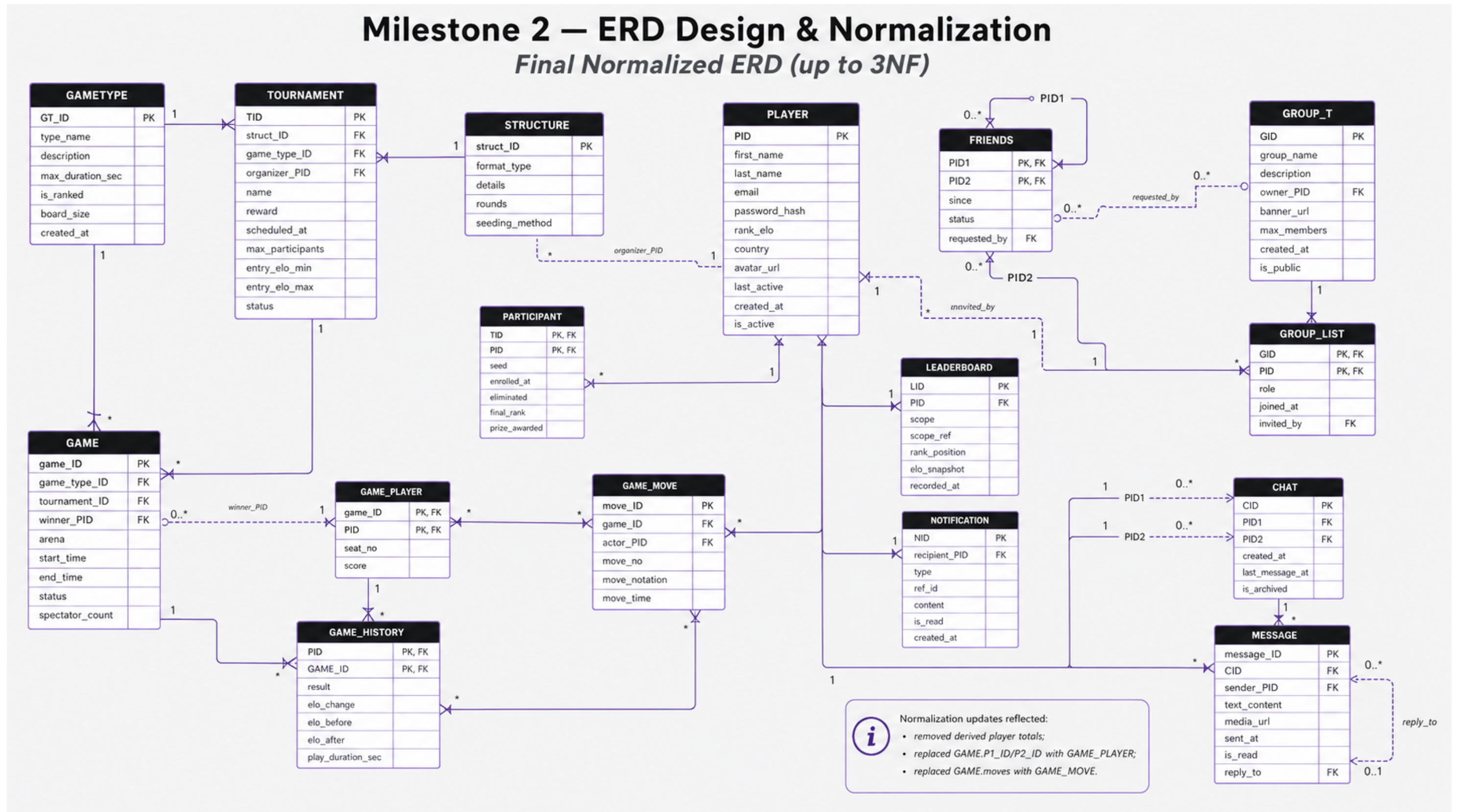


Figure 1. Final normalized ERD for the gaming/tournament platform database.

Final Normalized Schema Summary

The database is organized around players, game types, tournament structures, tournaments, games, gameplay activity, social relationships, groups, chat, messages, notifications, and leaderboard snapshots.

Table	Primary Key	Purpose / Main Dependencies
player	PID	Stores registered users, profiles, Elo rank, and account status.
gametype	GT_ID	Stores game modes and rule-level settings used by tournaments and games.
structure	struct_ID	Stores tournament formats and seeding methods.
tournament	TID	Depends on structure, gametype, and player organizer.
participant	TID + PID	Junction table between tournament and player.
game	game_ID	Depends on gametype, optional tournament, and optional winner player.
game_player	game_ID + PID	Junction table between game and player; replaces fixed P1/P2 columns.
game_move	move_ID	Stores one move per row; replaces storing a move list inside game.
game_history	PID + GAME_ID	Stores per-player result, Elo change, and duration for each game.
group_t	GID	Stores player-created groups and owner relationship.
group_list	GID + PID	Junction table between group_t and player with role and inviter.
friends	PID1 + PID2	Self-referencing player friendship/request relationship.
leaderboard	LID	Stores rank snapshots by global, game type, tournament, or country scope.
notification	NID	Stores system notifications delivered to players.
chat	CID	Stores one conversation between two players.
message	message_ID	Stores chat messages and optional reply-to self-reference.

Normalization Walkthrough: 1NF to 3NF

The schema was reviewed formally through First Normal Form, Second Normal Form, and Third Normal Form. The goal was to remove repeating groups, partial dependencies, transitive dependencies, and redundant/derived data while preserving meaningful relationships for the application.

Normal Form	Issue Reviewed	Change Made and Justification
1NF	Each table must contain atomic values, no repeating groups, and no multi-valued fields. The original game design contained fixed player columns such as P1_ID and P2_ID, and the game table also included a moves attribute that could store multiple moves in one field.	The fixed player columns were replaced with game_player, where each row stores one player in one game. The multi-valued moves attribute was replaced with game_move, where each row stores one move with move number, actor, notation, and time. This makes every attribute atomic and allows games to support more than two participants if needed.
2NF	Tables with composite primary keys must not have attributes that depend on only part of the key. This was especially important for participant, game_player, game_history, group_list, and friends.	Each composite-key table was checked so non-key attributes depend on the entire composite key. For example, participant.seed and participant.final_rank depend on the specific tournament-player pair, not only the tournament or only the player. game_player.seat_no and score depend on the specific game-player pair. group_list.role depends on the group-player membership pair. This removes partial dependency risk.
3NF	Non-key attributes must not depend on other non-key attributes, and derived values should not be stored when they can be calculated from transactional tables. The original player design included derived totals such as wins, losses, and draws.	Derived player totals were removed from player because they can be calculated from game_history. Tournament format details were kept in structure instead of being repeated inside tournament. Game type details were kept in gametype instead of being repeated in game or tournament. This prevents update anomalies and keeps facts stored in only one place.
Duplicate / Redundancy Review	The schema was checked for duplicated columns, overlapping relationship attributes, and repeated data across tables.	Redundant fixed player slots were removed from game and replaced by game_player. Repeated move data was moved into game_move. Player totals were not stored in player. Unique constraints were planned for natural identifiers such as player.email, gametype.type_name, and group_t.group_name. This keeps data consistent and reduces duplicate records.

Normalization Result

- All tables have primary keys and all relationship tables use meaningful composite keys where appropriate.
- Many-to-many relationships are resolved through junction tables: participant, game_player, group_list, friends, and game_history.
- Transitive and derived values are avoided; reports such as total wins, losses, and ranking summaries can be generated using joins or aggregate queries.
- The final ERD and DDL scripts match the normalized structure.

Project-Specific Dataflow Description

Data enters the system through user registration, game setup, tournament creation, gameplay, group/social activity, chat messages, and system notifications. Parent/reference data is created first, and dependent activity data is created afterward.

Stage	Tables Receiving Data	How Data Moves Through the Database
1. Reference and user setup	player, gametype, structure	Users register into player. Admins or system setup processes create game types and tournament structures. These tables must exist before tournaments and games can reference them.
2. Tournament creation and enrollment	tournament, participant	A tournament references its game type, format structure, and organizer player. Players enroll through participant, which links each player to a tournament using TID + PID.
3. Gameplay activity	game, game_player, game_move, game_history	A game references the game type, optional tournament, and winner. game_player records who played. game_move records each move in order. game_history records the outcome and Elo changes for each player in the game.
4. Group and friendship activity	group_t, group_list, friends	Players create groups in group_t. Membership and roles are stored in group_list. Friend requests and friendship statuses are stored in friends as a self-referencing player relationship.
5. Communication and notifications	chat, message, notification	Chat records one conversation between two players. Message stores each chat message and optional reply references. Notification stores system-generated alerts such as friend requests, tournament invites, game results, group invites, rank updates, and chat messages.
6. Outputs	leaderboard, reports, exports, query results	Ranking reports come from leaderboard and player. Match history comes from player, game_player, game, and game_history. Tournament standings come from tournament, participant, game, and game_history. Social reports come from group_t, group_list, and friends. Chat feeds come from chat and message.

Data Preparation Summary

Because no production dataset was provided, structured synthetic data was generated for all core tables. The dataset uses realistic IDs, dates, statuses, Elo values, relationships, and optional fields. Primary keys and composite keys were generated without duplicates. Foreign-key values were generated from existing parent IDs to preserve referential integrity.

Table	Rows	Table	Rows	Table	Rows	Table	Rows
player	100	gametype	50	structure	50	tournament	60
participant	100	game	50	game_player	100	game_move	100
game_history	100	group_t	50	group_list	100	friends	80
leaderboard	100	notification	100	chat	50	message	100

GitHub Repository and Submission Checklist

The GitHub repository should contain one clear milestone commit for each stage. The PDF upload and repository should match the final normalized schema.

Requirement	Repository File / Evidence	Status to Confirm
Updated ERD diagram	erd_design_and_normalization_overview.png or equivalent final ERD image	Confirm diagram matches DDL tables and relationships.
Normalization document with 1NF to 3NF justifications	Normalization markdown/PDF/DOCX document or this PDF content	Confirm each normal form includes issue, change, and reason.
Dataflow description	README_Dataflow.md and this PDF dataflow section	Confirm it is specific to the gaming/tournament platform.
Cleaned or synthetic CSV files	CSV folder with one file per table	Confirm 50-100 rows per core table and realistic relational values.
DDL scripts	milestone4_ddl.sql	Confirm CREATE TABLE statements, PKs, FKs, NOT NULL, UNIQUE, CHECK, and indexes are included.
DML scripts and validation output	01_insert_data.sql, 02_update_delete_demo.sql, 03_validation_queries.sql, 04_validation_output_commented.sql	Confirm UPDATE, DELETE, counts, NULL checks, and FK integrity checks are included.
Commit history	GitHub commit log	Confirm both group members contributed commits.

Suggested Commit History

Milestone	Suggested / Required Commit Message
M2	M2: ERD normalized and documentation added
M3	M3: Dataset preprocessing and dataflow added
M4	M4: DDL scripts added, EER diagram verified
M5	M5: Data populated validation queries added

Final Submission Notes

- Rename the PDF using the exact required format after replacing GroupName with the real group name.
- Paste the GitHub repository link on the cover page before uploading to Classroom.
- Verify that both members have visible commits in the GitHub history.
- Run the DDL and DML scripts in MySQL Workbench and keep validation screenshots or the commented validation SQL file in the repository.